

群衆雑音下のラグビー中継音声からのレフェリー笛検出

— エネルギーベース手法の限界と音色ベース分類による打破 —

野中哲 *

2026-06-17

要旨

ラグビー試合の中継音声から、レフェリーが吹く笛の時刻を自動検出する課題に取り組んだ。笛は約 2 kHz の狭帯域トーンだが、観客の歓声・チャント・ボールを蹴る音などの強い雑音に埋もれるため、単純なエネルギー検出では識別が難しい。本研究では、(1) 背景の取り方が相補的な 2 つのエネルギーベース検出器とその合議アンサンブル、(2) 候補のスペクトログラムから音色特徴を抽出して笛/非笛を判別するロジスティック回帰分類器、(3) 検出器の候補だけを効率的にラベル付けする支援ツール、を構築した。1 試合 (前半) を全編人手でラベル付けて得た笛 47 個を正解とすると、エネルギーベースのアンサンブルは適合率・再現率の調和平均 F1 で約 56% が上限であった。一方、音色特徴 (立ち上がりの鋭さ・倍音比・低域比など) を用いた分類器で候補を再ランクすると、5 分割交差検証で **F1 は 56% から 72% へ向上した**。さらに前半で学習した分類器を未学習の後半に適用したところ、後半でも **F1 74% → 91%** と汎化した。本稿は手法・実験・知見を報告するとともに、「音の大小」では分離できない笛を「音色」で分離するという、人間の聴覚に着想を得た設計指針を示す。

キーワード: 音響イベント検出, スペクトログラム, 笛検出, アンサンブル, 音色分類, 弱教師

1. はじめに

スポーツ映像の自動編集・ハイライト生成では、試合の節目を示すイベント時刻の取得が重要である。ラグビーにおいてレフェリーの笛は反則・得点・前後半の開始終了など多くの局面を標識する有力な手がかりとなる。しかし中継音声の中の笛は、

- 観客の歓声・ざわめき (広帯域・非定常)
- チャント・声援 (調波構造をもつ)
- ボールを蹴る音・拍手 (瞬発的)

といった強い背景音に埋もれる。特に主審がマイクから遠い場面や観客が盛り上がる場面では、笛のエネルギーが背景と同程度になり、振幅情報だけでは識別が困難になる。

本研究の貢献は次の通りである。

1. 背景推定の異なる 2 つのエネルギーベース検出器と、その合議によるアンサンブルを設計した。
2. エネルギーベース手法が $F1 \approx 56\%$ で頭打ちになることと、その原理的理由を実測で示した。

* 神奈川不惑クラブ

3. スペクトログラムから抽出した音色特徴による分類器で候補を再ランクし、頭打ちを打破した (交差検証 F1 56 → 72%)。さらに別ハーフへの汎化 (F1 74 → 91%) を確認した。
4. 音声領域の前処理 (調波・打楽器分離; HPSS) は本課題では有効でないという否定的結果を 実測で示した。
5. 検出器の候補だけを音声 + スペクトログラム付きでレビューし高速にラベル付けする支援ツールを構築し、ラベリングにおける実践的知見 (クリップはエネルギーピークに中心を合わせるべき) を得た。

2. データ

公式リーグ戦 1 試合の中継動画を用いた。音声は 48 kHz・モノラル・16 bit PCM に変換した。

ハーフ	長さ	笛 (正解)	備考
前半	44 分 10 秒 (2647 s)	47	全編リスニングでラベル付け
後半	47 分 28 秒 (2848 s)	41(候補中)	検出器候補をレビューでラベル付け

前半は人手で全編を聴いて笛を書き起こした。当初 48 個と記録したが、後述するクリップ精査により 1 個が数え過ぎと判明し、最終的に**笛 47 個**とした。加えて、検出器が誤検出しやすい「笛でない強いトーン」2 個 (例: 29:55) をハード負例として記録した。

後半は本研究で構築した支援ツールにより、検出器が出した候補 118 個をレビューして**笛 41 / 非笛 77**に分類した。後半は全編リスニングを行っていないため、後半の正解は「候補に含まれる笛」に限られる (2. の限界として後述)。

3. 手法

3.1 エネルギーベース検出

笛は基本波 $f_0 \approx 1.7\text{--}2.5\text{ kHz}$ とその倍音をもつ、短い持続的な狭帯域トーンである。音声を STFT でスペクトログラム化し、フレームごとに以下を評価して笛らしさを数値化する。

- **トーン性:** 笛帯域でのピークエネルギーの集中度 (ピーク/帯域和)
- **倍音の同時性:** f_0 と $2f_0$ 帯域のエネルギーの同時の強さ
- **背景に対するコントラスト:** ピークを背景レベルで割った突出度
- **持続:** 一定時間トーンが続くか (瞬発音の除去)

背景レベルの取り方を変えた 2 つの「専門家」を用意した。

- **v1:** 12 kHz にダウンサンプルし、コントラストの分母に**低域 (150–1300 Hz) の群衆レベル**を用いる。持続をやや長めに要求する。明瞭で持続的な笛に強い (高精度) が、短い笛や群衆が大きい場面の弱い笛を取り逃す。
- **v2:** 24 kHz で解析し、コントラストの分母に**笛帯域そのものの時間方向の局所背景**を用いる。持続要求を短くする。人間の聴覚の周波数選択性 (低域の群衆は 2 kHz の笛をマスクしない) を反映し、短い/弱い笛を拾えるが、瞬発的な縦縞ノイズ等の誤検出が増える。

各検出器はフレームスコアの局所ピークをイベント化して候補を出す。両者の候補を、各検出器での「(候補数の上限 - 順位)」を加算した**合議スコア**でグローバルに再ランクする (両方に出る候補ほど高信頼)。これを本番の候補生成器 (アンサンブル) とする。

3.2 評価方法と正解データ

検出時刻が正解の笛時刻と ± 4 s 以内で一致すれば真陽性 (TP) とする貪欲 1 対 1 照合を用い、上位 N 件を採ったときの適合率 (precision) ・再現率 (recall) ・F1 を曲線として評価する。あわせて、候補中に含まれる正解笛の割合を**再現率上限**と定義する (候補に無い笛はどんな再ランクでも拾えないため)。

3.3 前処理の検討: HPSS(否定的結果)

スペクトログラムを時間方向メディアン (調波=笛) と周波数方向メディアン (打楽器=ポール音) に分離し、調波成分を強調した音声を再構成して検出器に入力した。しかし 2 通りのカーネル設定のいずれでも、**再現率上限が悪化し (90% → 69~77%)、F1 は改善しなかった**。原因は、(i) 音声再構成 (逆 STFT) が弱い笛を損なうこと、(ii) v2 が既に帯域内の局所背景比で適応正規化しており前処理と重複すること、(iii) 声・チャントも調波構造をもつため笛と分離できないこと、である。よって本課題では音声領域の前処理は採用しない。

3.4 音色ベース分類器

エネルギーベースの頭打ちを打破するため、候補のスペクトログラム (± 0.4 s 窓) から音色特徴 11 次元を抽出し、笛/非笛をロジスティック回帰で判別して候補を再ランクする。特徴は突出度・トーン性・倍音比 ($f_0, 2f_0, 3f_0$) ・ピッチ安定性・低域比・高域比・持続・立ち上がりの鋭さ・スペクトル平坦度などである。学習は標準化 + L2 正則化、評価は 5 分割交差検証の out-of-fold スコアで行い、学習データへの過適合を排した。

3.5 教師データ作成支援ツール

別試合のラベル付けを効率化するため、検出器の候補 (高再現) だけをレビューする自己完結 HTML を生成するツールを構築した。各候補に音声クリップとスペクトログラムを埋め込み、キーボードで笛/非笛/保留を高速にラベル付けできる。ローカルサーバへの自動保存により書き出し操作を不要にし、進捗ミニマップ・未/済の色分け・未ラベルへのジャンプを備える。重要な改良として、レビュー用クリップを候補時刻ではなく**近傍の笛帯域エネルギーピークに中心を合わせ直す処理**を導入した (3.5 の知見、4.4)。

4. 実験結果

4.1 検出器の性能 (前半, 笛 47)

各検出器の再現率上限と、アンサンブル合議ランキングの動作点を表に示す。

検出器	再現率上限
v1 単独	58%
v2 単独	79%
アンサンブル (v1 $\cup$ v2)	約 90%

N(上位件数)	precision	recall	F1
15	100%	31%	48%
30	73%	46%	56%(最良)
70	44%	65%	53%
120	36%	90%	51%

上位 15 件は誤検出ゼロである一方、F1 は約 56% で頭打ちとなる。再現率上限は約 90% で、**残り 4 個の笛 (1:57, 11:35, 12:39, 23:57) はどの設定でも候補に上がらない**。これらは背景に対する帯域内突出が小さく (例: 1:57 は突出 1.7 倍。検出可能な笛は 3~6 倍)、エネルギー情報だけでは背景と分離できない。

4.2 前処理 (HPSS)

入力	再現率上限	F1 最良
前処理なし	90%	56%
HPSS (KT=7,KF=9)	69%	58%
HPSS (KT=3,KF=13)	77%	56%

前処理は再現率上限をむしろ下げ、F1 を改善しなかった (3.3)。

4.3 音色分類器 (前半, 5 分割交差検証)

N	baseline P/R/F1	分類器 P/R/F1
30	73 / 46 / 56	90 / 56 / 69
40	57 / 48 / 52	78 / 65 / 70
最良	56%	72%

F1 は 56% → 72%(+16 ポイント) に向上した (図 1, 図 2)。回帰係数の大きい特徴は、立ち上がりの 鋭さ (onset, 最大の正係数)、低域比 (群衆の少なさ, 負係数)、倍音比、持続であり (図 3)、「笛らしい音色」を学習していることを示す。再現率上限は候補集合に依存するため 90% のままで、分類器はあくまで既存候補の **ランキング品質** を改善する。

4.4 汎化: 前半で学習→後半でテスト

前半の笛で学習した分類器を、未学習の後半候補 (笛 41/非笛 77) に適用した。

後半での手法	precision	recall	F1
baseline(合議スコア順)	75%	73%	74%
分類器 (前半で学習→後半に適用)	92%	90%	91%

F1 74% → 91%。音色特徴が試合 (ハーフ) をまたいで有効であることを示す (図 4)。

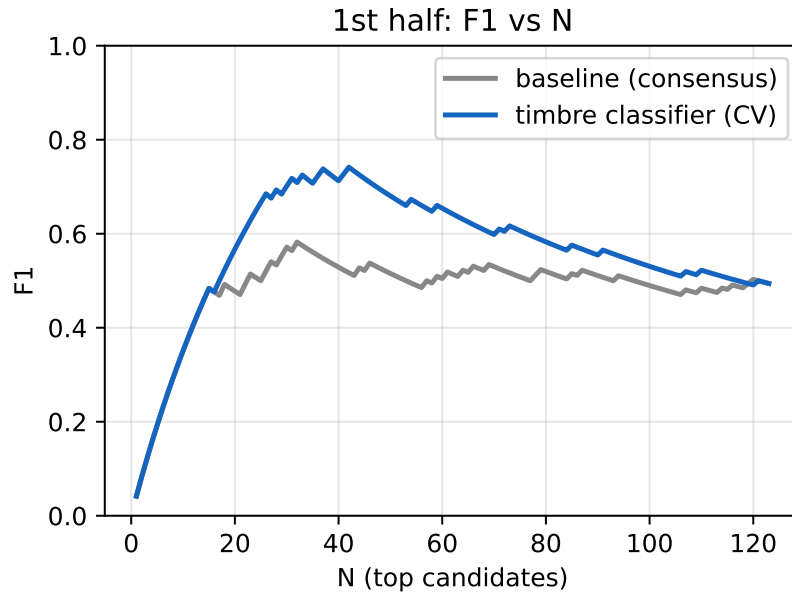


図1 前半における F1 の比較。合議スコア順 (baseline) に対し、音色分類器 (交差検証) はあらゆる上位 N 件で F1 が高い。

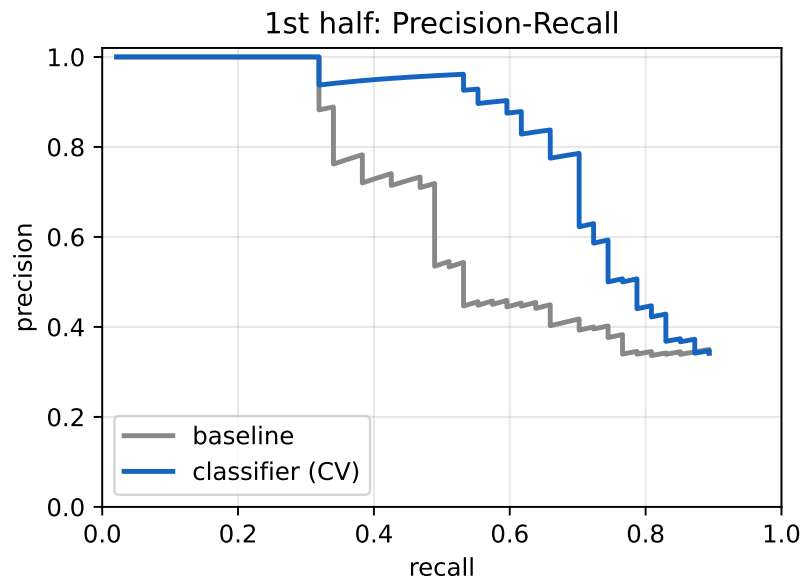


図2 前半の適合率-再現率曲線。分類器 (交差検証) は baseline を一貫して上回る。

加えて、支援ツールの設計に関する知見を得た。候補だけのレビューでは前半の笛が 30 個しか拾えず、全編リスニングの 47 個と 13 個食い違った。その 13 個を「正解時刻に中心を合わせたクリップ」で聴き直すと **12 個は笛**であった。原因は候補の時刻が実際の笛と数秒ずれ、候補中心のクリップが笛を外していたことである。残り 1 個は数え過ぎであった。この知見から、レビュー用クリップを近傍のエネルギーピークに中心合わせする処理を導入した (3.5)。

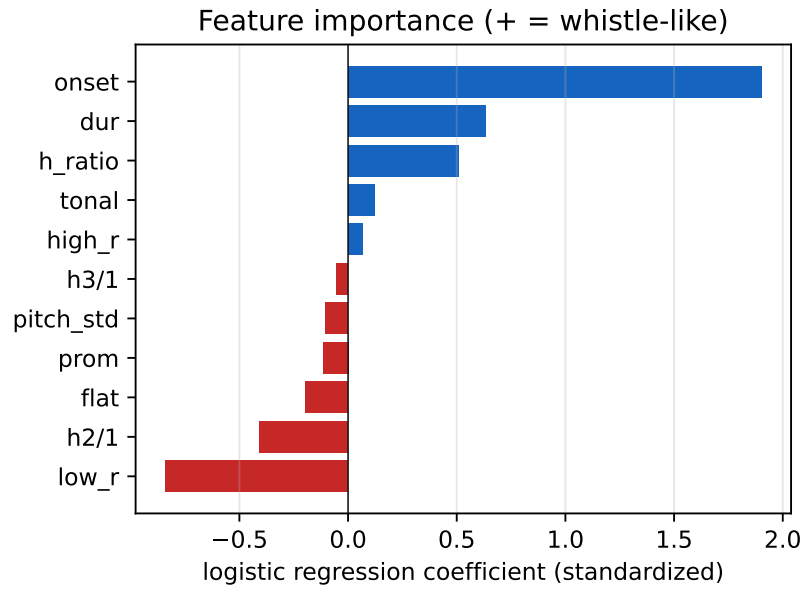


図3 ロジスティック回帰の標準化係数。立ち上がりの鋭さ (onset) が最も笛らしさに寄与し、低域比 (low_r=群衆の大きさ) は負に効く。

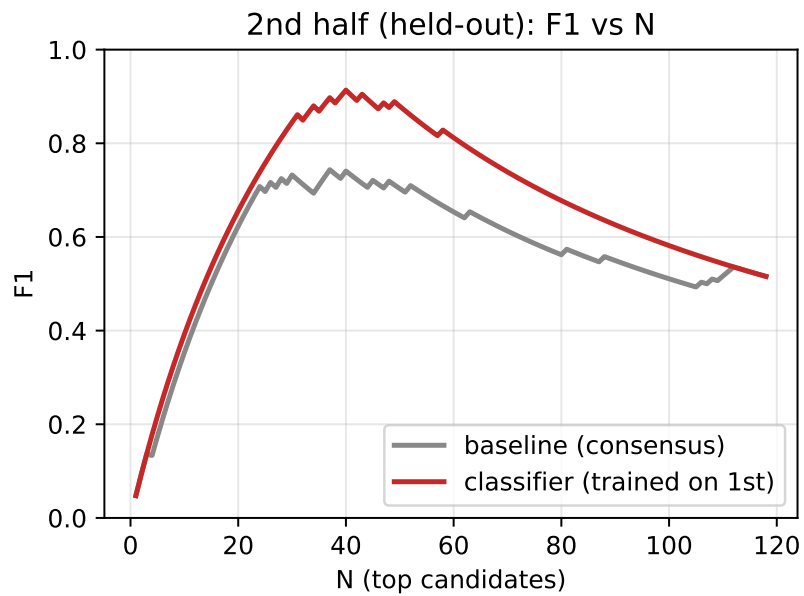


図4 後半 (未学習データ) での F1。前半で学習した分類器が baseline を上回り、F1 は 74% → 91% に向上する。

5. 考察

5.1 なぜエネルギーでは頭打ちか

エネルギーベース手法は「笛帯域がどれだけ大きく突出したか」を測る。しかし観客の声やチャントは笛と同程度の音量・同様の調波構造をもつため、振幅・突出度だけでは笛と分離できない。これが $F1 \approx 56\%$ の壁の本質である。

5.2 人間の聴覚との対応

人間は群衆が大きい場面でも 1:57 の弱い笛を明瞭に聞き取る。これは振幅比ではなく、(i) 周波数選択性 (臨界帯域: 低域の群衆は 2 kHz の笛をマスクしない)、(ii) 倍音のグループ化 (聴覚情景分析)、(iii) 立ち上がり・時間微細構造への鋭敏さ、(iv) 音色・パターン認識による。本研究の v2 は (i) を、音色分類器は (iv) を計算的に近似したものと位置づけられる。立ち上がり (onset) が分類の最重要特徴であった点は (iii) とも整合する。

5.3 限界

- 検出不能な 4 個の笛は候補に上がらないため、分類器でも回収できない。候補生成器 (検出器) の感度向上が別途必要である。
- 前半/後半は同一試合 (同じ主審・笛・録音環境) であり、別主審・別会場・別放送への汎化は未検証である。
- 後半の正解は候補ベースであり、全編リスニングを行っていないため後半の絶対再現率は未確定である。報告した汎化はランキング品質の改善である。

6. 結論と今後

群衆雑音下の笛検出において、エネルギーベース手法は $F1 \approx 56\%$ で頭打ちになること、そして音色ベースの分類器がこの壁を打破し (交差検証 $F1\ 72\%$)、別ハーフへ汎化する ($F1\ 91\%$) ことを実測で示した。否定的結果として、音声領域の HPSS 前処理は本課題で有効でないことも示した。今後は、(1) 分類器を本番パイプラインへ組み込み (モデル保存→新規試合へ適用)、(2) 複数試合・複数会場のデータによる真の汎化検証、(3) 検出不能な弱い笛を拾うための候補生成器の改良、(4) 全編リスニングによる絶対再現率の確立、に取り組む。

付録: 再現性

本研究の実装は単一リポジトリ <https://github.com/anonaka/fue> にまとめ、Python(numpy) と ffmpeg/ffprobe のみで動作する。Python 依存パッケージは uv で管理し (pyproject.toml/uv.lock)、uv run で各スクリプトを実行すれば同一環境を再現できる。主なスクリプトは、検出器 (detect_whistle.py, _v1.py, _v2.py)、評価 (evaluate.py)、音色分類器 (classify.py, 汎化検証 classify_generalize.py)、前処理実験 (preprocess.py)、ラベル付け支援 (make_review.py, review_server.py)、マーカー生成 (make_chapters.py) である。正解データは labels/ground_truth.tsv(前半), labels/ground_truth_2nd.tsv(後半) に TSV 形式で保持する。